

Moderner, solider und testbarer ABAP Code

Agenda

1. Einleitung
2. Muster
3. ABAP Unit
4. Tipps
5. Zusammenfassung

Vorstellung



Björn Schulz

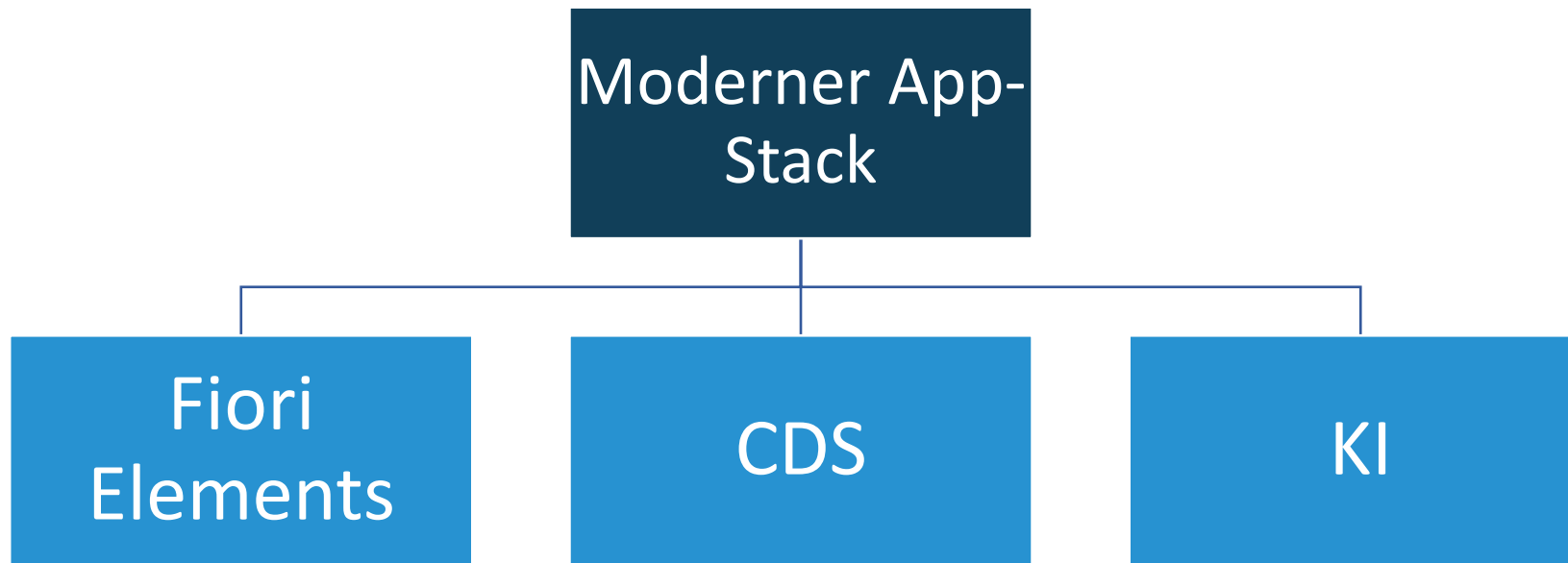
- SAP Developer
- Über 15 Jahre ABAP
- Content Creator (SwH)
- DSAG Sprecher, SAP Mentor

WHY
NOT?

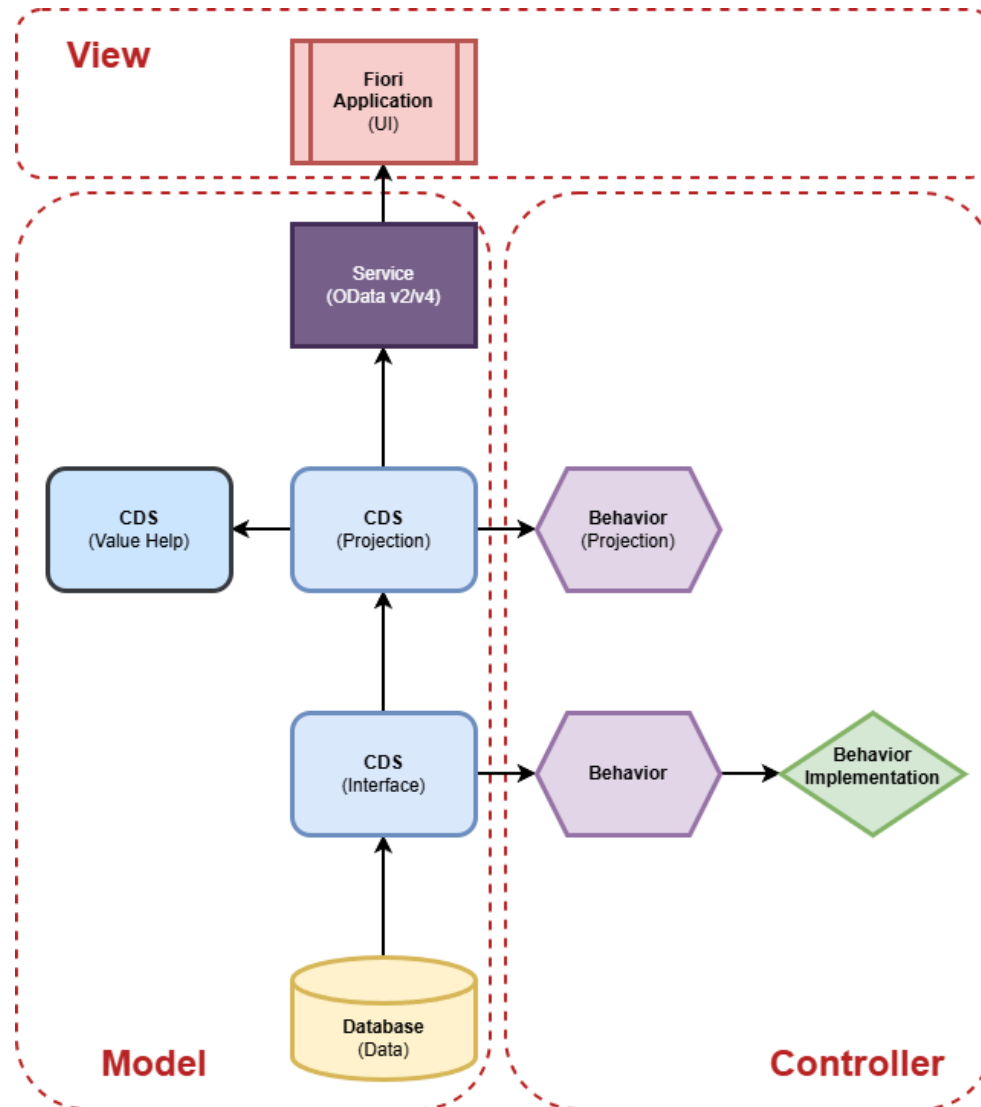
Einleitung

Einleitung

Warum eigentlich noch ABAP?



Einleitung



ABAP und
ABAP Unit

Clean ABAP

Struktur und
Kapselung

Einleitung

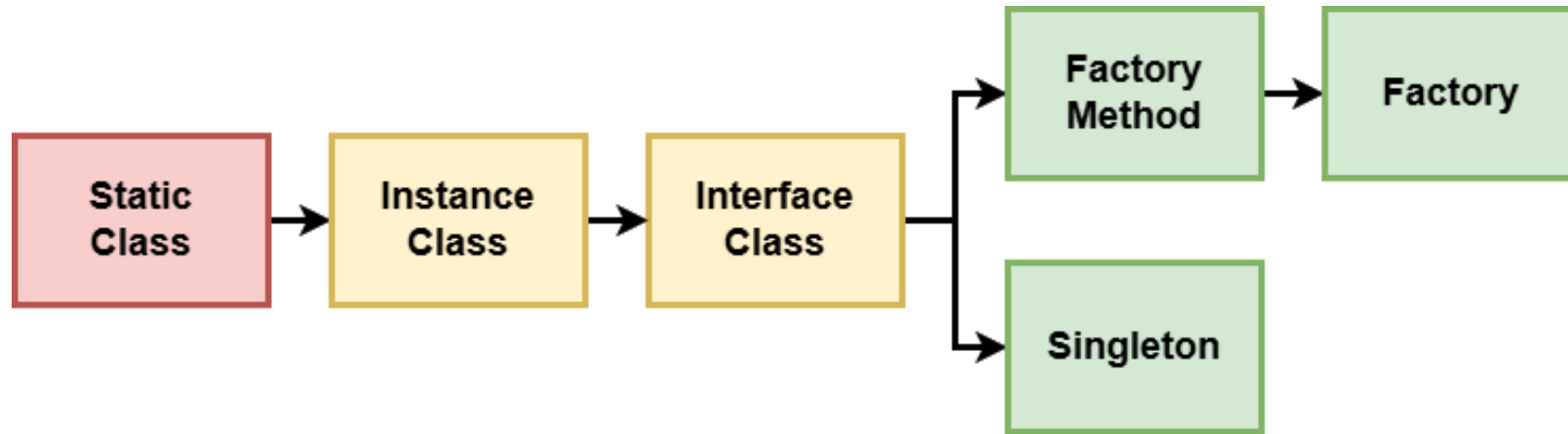
Was erwartet dich?





Muster

Muster



Aufbau

- Zeitstempel erzeugen, speichern
- Zugriff über ein Attribut

Muster

Statische Klasse

Vorteile

- Implementierung übersichtlich
- Wenige Artefakte

Nachteile

- Nur eine Instanz möglich

```
CLASS zcl_bs_bc26_static_class DEFINITION
  PUBLIC FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    CLASS-DATA timestamp TYPE timestamp1 READ-ONLY.

    CLASS-METHODS get_timestamp
      RETURNING VALUE(result) TYPE timestamp1.

    CLASS-METHODS save_timestamp.
ENDCLASS.

CLASS zcl_bs_bc26_static_class IMPLEMENTATION.
  METHOD get_timestamp.
    GET TIME STAMP FIELD result.
  ENDMETHOD.

  METHOD save_timestamp.
    timestamp = get_timestamp( ).
  ENDMETHOD.
ENDCLASS.
```

Muster

Instanz Klasse

Vorteile

- Mehrere Instanzen gleichzeitig möglich

Nachteile

- Keine verschiedene Ausprägung der gleichen Klasse

```
CLASS zcl_bs_bc26_instance DEFINITION
    PUBLIC FINAL
    CREATE PUBLIC.

    PUBLIC SECTION.
        DATA timestamp TYPE timestamp1 READ-ONLY.

        METHODS get_timestamp
            RETURNING VALUE(result) TYPE timestamp1.

        METHODS save_timestamp.
ENDCLASS.
```

```
CLASS zcl_bs_bc26_instance IMPLEMENTATION.
    METHOD get_timestamp.
        GET TIME STAMP FIELD result.
    ENDMETHOD.

    METHOD save_timestamp.
        timestamp = get_timestamp( ).
    ENDMETHOD.
ENDCLASS.
```



Muster

Interface Klasse

Vorteile

- Wiederverwendbarer Bauplan der Klasse

Nachteile

- Aufwand bei Inline Deklaration

Interface

```
INTERFACE zif_bs_bc26_interface
PUBLIC.

DATA timestamp TYPE timestamp1 READ-ONLY.

METHODS get_timestamp
RETURNING VALUE(result) TYPE timestamp1.

METHODS save_timestamp.
ENDINTERFACE.
```

Klasse

```
CLASS zcl_bs_bc26_interface DEFINITION
PUBLIC FINAL
CREATE PUBLIC.

PUBLIC SECTION.
INTERFACES zif_bs_bc26_interface.

ALIASES timestamp FOR zif_bs_bc26_interface~timestamp.
ENDCLASS.

CLASS zcl_bs_bc26_interface IMPLEMENTATION.
METHOD zif_bs_bc26_interface~get_timestamp.
GET TIME STAMP FIELD result.
ENDMETHOD.

METHOD zif_bs_bc26_interface~save_timestamp.
timestamp = zif_bs_bc26_interface~get_timestamp( ).
ENDMETHOD.
ENDCLASS.
```



Muster

Factory Methode

Vorteile

- Direkte Verwendung der erzeugten Instanzen
- Verkettung von Aufrufen

Nachteile

- Abhängigkeit zur implementierenden Klasse

```
CLASS zcl_bs_bc26_factory_method DEFINITION
    PUBLIC FINAL
    CREATE PRIVATE.

    PUBLIC SECTION.
        INTERFACES zif_bs_bc26_interface.

        CLASS-METHODS create
            RETURNING VALUE(result) TYPE REF TO zif_bs_bc26_interface.
ENDCLASS.

CLASS zcl_bs_bc26_factory_method IMPLEMENTATION.
    METHOD create.
        RETURN NEW zcl_bs_bc26_factory_method( ).
    ENDMETHOD.

    METHOD zif_bs_bc26_interface~get_timestamp.
        GET TIME STAMP FIELD result.
    ENDMETHOD.

    METHOD zif_bs_bc26_interface~save_timestamp.
        zif_bs_bc26_interface~timestamp = zif_bs_bc26_interface~get_timestamp( ).
    ENDMETHOD.
ENDCLASS.
```



Muster

Factory

Vorteile

- Entkopplung der eigentlichen Implementierung

Nachteile

- Anzahl der Artefakte

Factory

```
CLASS zcl_bs_bc26_factory DEFINITION
  PUBLIC ABSTRACT FINAL
  CREATE PRIVATE.

  PUBLIC SECTION.
    CLASS-METHODS create
      RETURNING VALUE(result) TYPE REF TO zif_bs_bc26_interface.
ENDCLASS.
```

```
CLASS zcl_bs_bc26_factory IMPLEMENTATION.
  METHOD create.
    RETURN NEW zcl_bs_bc26_private( ).
  ENDMETHOD.
ENDCLASS.
```

Klasse

```
CLASS zcl_bs_bc26_private DEFINITION
  PUBLIC FINAL
  CREATE PRIVATE
  GLOBAL FRIENDS zcl_bs_bc26_factory.

  PUBLIC SECTION.
    INTERFACES zif_bs_bc26_interface.
ENDCLASS.
```

```
CLASS zcl_bs_bc26_private IMPLEMENTATION.
  METHOD zif_bs_bc26_interface~get_timestamp.
    GET TIME STAMP FIELD result.
  ENDMETHOD.

  METHOD zif_bs_bc26_interface~save_timestamp.
    zif_bs_bc26_interface~timestamp = zif_bs_bc26_interface~get_timestamp( ).
  ENDMETHOD.
ENDCLASS.
```



Muster

Singleton

Vorteile

- Genau eine Instanz gewünscht
- Keine Speicherung nötig

Nachteile

- Mehr Aufwand für die Verwaltung der Instanz(en)

```
CLASS zcl_bs_bc26_singleton DEFINITION
PUBLIC FINAL
CREATE PRIVATE.

PUBLIC SECTION.
DATA timestamp TYPE timestamp1 READ-ONLY.

CLASS-METHODS get_instance
RETURNING VALUE(result) TYPE REF TO zcl_bs_bc26_singleton.

METHODS get_timestamp
RETURNING VALUE(result) TYPE timestamp1.

METHODS save_timestamp.

PRIVATE SECTION.
CLASS-DATA singleton TYPE REF TO zcl_bs_bc26_singleton.
ENDCLASS.

CLASS zcl_bs_bc26_singleton IMPLEMENTATION.
METHOD get_instance.
IF singleton IS NOT BOUND.
    singleton = NEW zcl_bs_bc26_singleton( ).
ENDIF.

RETURN singleton.
ENDMETHOD.

METHOD get_timestamp.
GET TIME STAMP FIELD result.
ENDMETHOD.

METHOD save_timestamp.
timestamp = get_timestamp( ).
ENDMETHOD.
ENDCLASS.
```



Muster

Beispiele

Muster	Beispiel
Statisch	CL_ABAP_CONTEXT_INFO
Instanz	CL_ABAP_GZIP_BINARY_STREAM
Interface	CL_API_JT_CHECK_BASE
Factory Methode	CL_SXML_STRING_READER
Factory	CL_IAM_BUSINESS_USER_FACTORY
Singleton	CL_SWF_TST_CONTAINER_SINGLETON

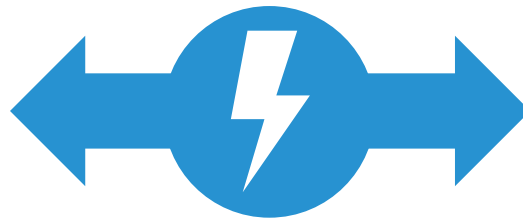
ABAP Unit

ABAP Unit



Kein Test

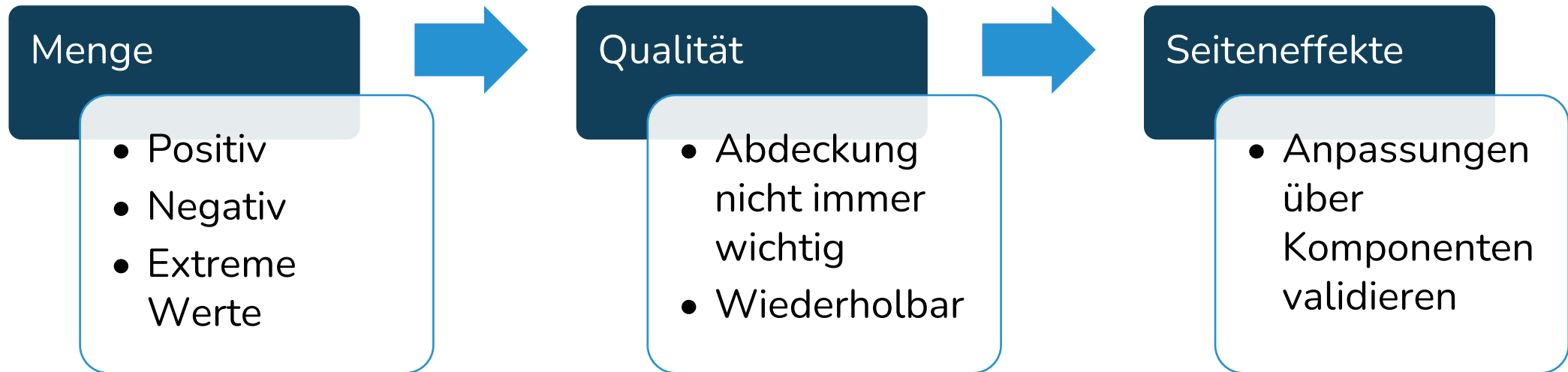
- Manuelle Prüfung der verschiedenen Code Bestandteile
- Risiko für nicht getesteten Code
- Angst vor Änderungen von bestehenden Logiken



Unit Test

- Validierung gegen Ergebnis direkt in der Entwicklung
- Sicherheitsnetz bei Änderungen
- Nachhaltung von Sonderfällen
- Dokumentation der Funktionsweise

ABAP Unit



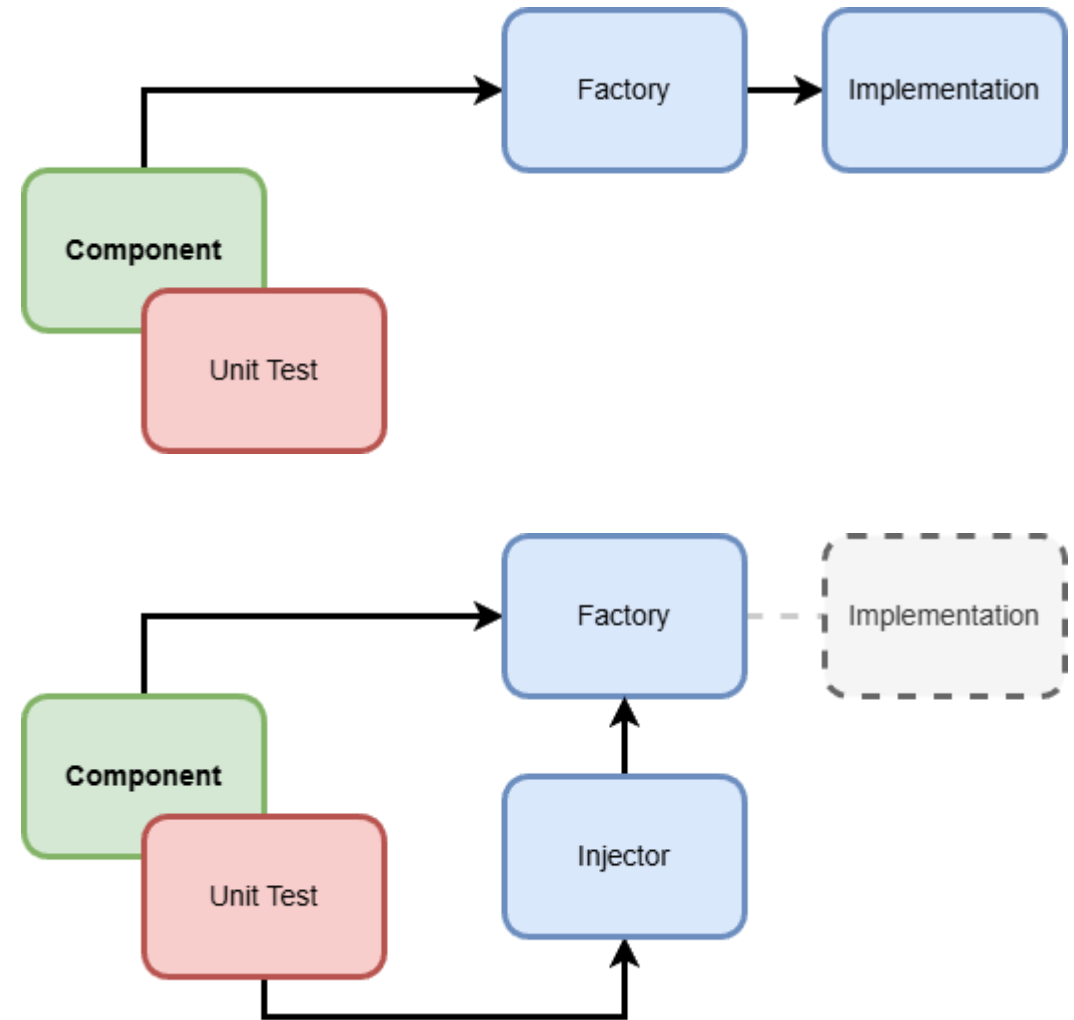
ABAP Unit

Szenario

- Verwendung der Factory
- Erstellung Unit Test auf der verwendenden Komponente

Injector

- Möglichkeit die Implementierung austauschen
- Keine Anpassung des Codes





Tipps

Tipps

Tools und Informationen

Avoid obsolete language elements

[Clean ABAP > Content > Language > This section](#)

When upgrading your ABAP version, make sure to check for obsolete language elements and refrain from using them.

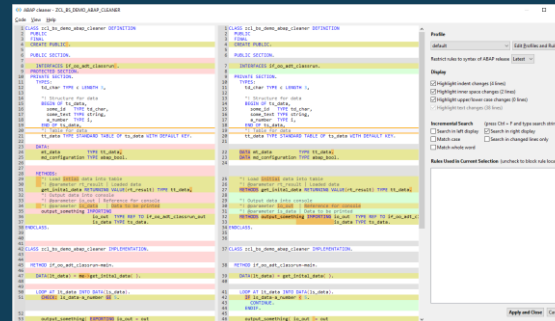
For example, the @-escaped "host" variables in the following statement make a little clearer what's a program variable and what's a column in the database,

```
SELECT *  
FROM spfl1  
WHERE carrid = @carrid AND  
connid = @connid  
INTO TABLE @itab.
```

as compared to the [obsolete unescaped form](#)

```
SELECT *  
FROM spfl1  
WHERE carrid = carrid AND  
connid = connid  
INTO TABLE itab.
```

Clean ABAP



ABAP Cleaner

The screenshot shows the Code Pal for ABAP tool interface. It displays a list of code snippets with various annotations and search options. The table below summarizes the findings shown in the interface.

Description	Line Number	Object Type	Check
Number of attributes must be lower than 121 (13+ = 12)	1	CLAS	Number of Attributes
Prefer NEW to CREATE OBJECT	2 (CREATE)	CLAS	Prefer New to Create Object
'CALL METHOD' statement should not be used	4 (DO_IT_AGAIN)	CLAS	CALL METHOD Usage
CK_ROOT should not be used	5 (GET_UUID)	CLAS	CK_ROOT usage
1 public methods without interface	6	CLAS	Interface Missing
Consider calling the RETURNING parameter RESULT	14 (ZCL_CLEAN_UP_CLASS)	CLAS	Returning Name
Method DO_IT has a misleading name for boolean return type	16 (ZCL_CLEAN_UP_CLASS)	CLAS	Method Name Misleading for Boolean Return
Split methods instead of adding OPTIONAL parameter	2 (ZCL_EXTRACT_METHOD)	CLAS	Optional Parameters
Magic Number Violation - 255 is a Magic Number!	2 (SET_VALUE)	CLAS	Magic Number Usage
Magic Number Violation - 255 is a Magic Number!	3 (ADD_VALUE)	CLAS	Magic Number Usage
Consider calling the RETURNING parameter RESULT	8 (ZCL_MAGIC_NUMBER_UP_CLASS)	CLAS	Returning Name

Code Pal for ABAP

Tipps

My IDE Actions (MIA)

- Verwendung von IDE-Actions zur Anlage der Artefakte
- Klare Regeln und Einhaltung der Guidelines
- Automatisierung des Aufwands, auch ohne KI

The screenshot displays the SAP IDE interface during the execution of the 'Create new class' action. The 'Create new class' dialog is open, showing the following configuration:

- Package: * ZBS_DEMO_GENERATED
- Transport: * 759
- Prefix: * Z
- Name: * MIA_RUN
- Description: * MIA Runner
- Class: * ZCL_MIA_RUN
- Interface: ZIF_MIA_RUN
- Factory: ZCL_MIA_RUN_
- Injector:

The 'HTML Action Result' window shows the result of the action:

Result [759]

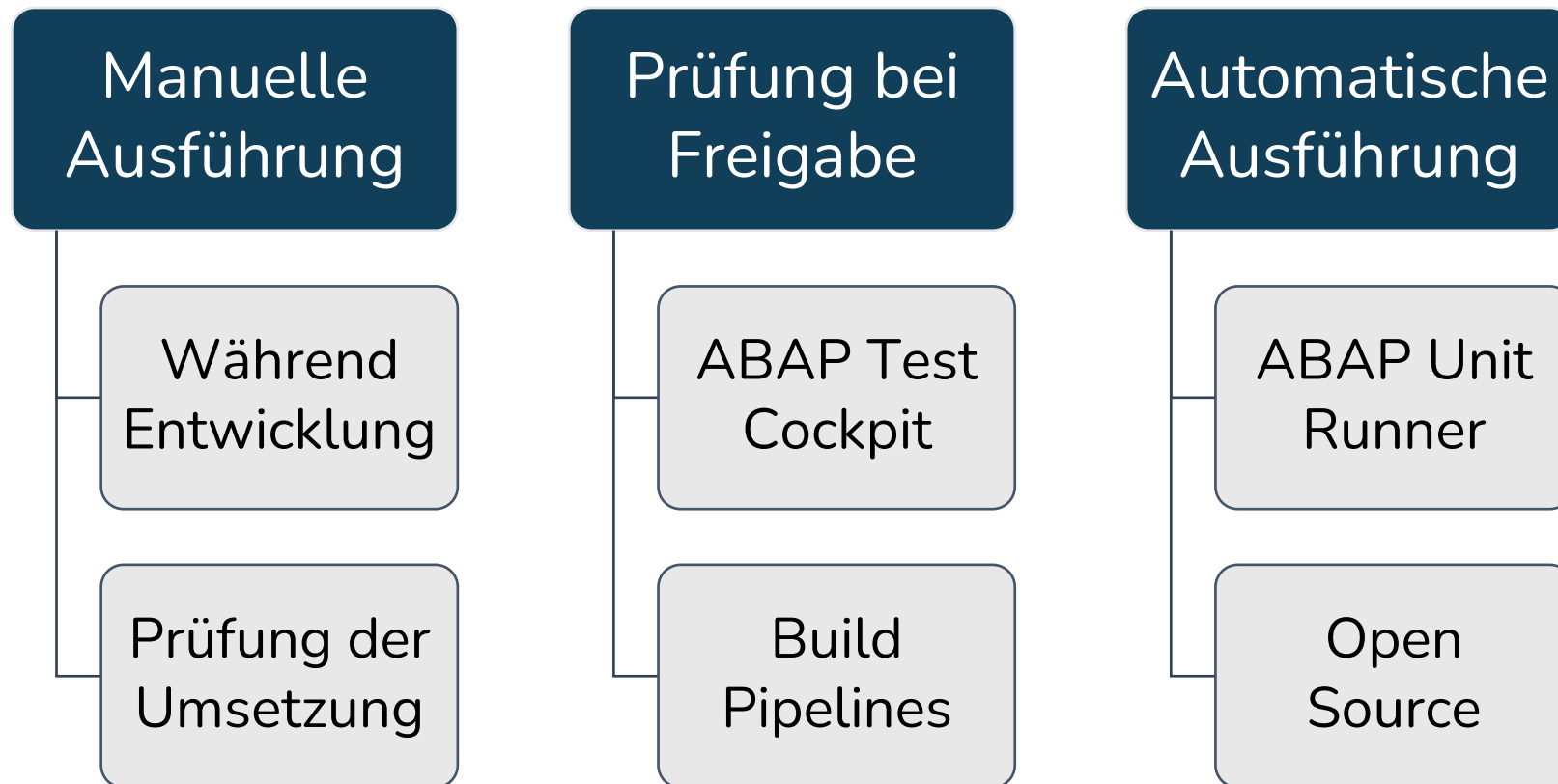
Object	Name
Interface	ZIF_MIA_RUN
Class	ZCL_MIA_RUN
Factory	ZCL_MIA_RUN_FACTORY

The 'Generated Objects' tree on the right shows the following structure:

- ZBS_DEMO_GENERATED (3) Generated Objects
 - Classes (2)
 - ZCL_MIA_RUN MIA Runner (Class)
 - ZCL_MIA_RUN_FACTORY MIA Runner (Factory)
 - Interfaces (1)
 - ZIF_MIA_RUN MIA Runner (Interface)

Tipps

Regelmäßige automatische Tests



Tipps

ABAP Unit Runner

Select Objects for Unit Tests

Select by Package: ☒
 Select by Program: ☐

Select by Package

Packages: 

With Subpackages: ☒

Exclude Selected Objects...: ☐

ABAP Unit

Overview

Variant: -
System: -
Client: -
Started by: -
Started on: 04.01.2024
Started at: 16:21:41 (CET)
Finished at: 16:21:49 (CET)

• Fatal Failures: 1
▲ Critical Failures: 7

Method Statistics

Packages	Programs	Test Classes	Test Methods	Passed	With Fatal	With Critical	With Tolerable	Skipped/No Exec.
6	12	12	46	30	1	6	0	1

Failures

Level	Test Class	Test Method	Description	Object Name	Type	Package
▲	LTC_LOOS	n.a.	Exception Error: <CXX_CA_LOG_ERROR>		CLAS	
▲	LCL_TST_DISTMODEL	ADD_DATA_TEST_MODEL	Critical Assertion Error: 'Failed to update model'		CLAS	
▲	LCL_TST_DISTMODEL	DELETE_DATA_TEST_MODEL	Critical Assertion Error: 'Failed to update model'		CLAS	
▲	LCL_TST_DISTMODEL	READ_DATA_VALID_MODEL	Critical Assertion Error: 'Failed to read model'		CLAS	
▲	LCL_TST_REMOTE	READ_DEST_OK	Critical Assertion Error: 'RFC destination is not valid'		CLAS	
▲	LCL_TST_REMOTE	READ_DEST_OK_SUB	Critical Assertion Error: 'RFC destination is not valid'		CLAS	
▲	LTC_EXTRACTION	n.a.	Runtime Error: <PFS>		CLAS	
▲	LTC_TABLE_TEST	FILL_SELECTION	Critical Assertion Error: 'Implement your first test here'		CLAS	

Packages

On-Premise | Private Cloud

Destinations

Destination:
Communication Arrangement:

Settings

General

Title:
Wait (Cycles):
Wait (Seconds):

Scope

Own: ☒
Foreign: ☒

Duration

Short: ☒
Medium: ☐
Long: ☐

Risk Level

Harmless: ☒
Dangerous: ☐
Critical: ☐

Selection

Packages:
Classes:

Notification

Send mail: ☐
Only send, when error: ☐
Sender:
Receiver:

E-Mail Settings

Send mail: ☐
Only send, when error: ☐
Sender:
Receiver:

General Informations

Title	System	Client	Execution User	Run Time	Timestamp	Failures	Errors	Skipped	Asserts	Tests
Testrun This	100			5.120	2025-02-06T19:05:24Z	5	1	0	5	47

Package: ZBS_DEMO_TESTS

ZCL_BS_DEMO_API_OUT [LTC_READ_ENTITY]

Tests	Failures	Errors	Skipped	Asserts	Timestamp	Time
3	1	1	0	1	2025-02-06T19:05:25Z	0.249

Method **Time** **Asserts**

CREATE_FAIL	0.186	0
READ_ENTITY	0.035	0
READ_LIST	0.027	1

ZCL_BS_DEMO_CDS [LTC_ZBS_C_COMPANYCODEOUT]

Tests	Failures	Errors	Skipped	Asserts	Timestamp	Time
1	0	0	0	0	2025-02-06T19:05:28Z	0.215

Open Source Project (Cloud)

Zusammenfassung

Zusammenfassung



Wichtigkeit

- Mit RAP und KI bleibt das Thema wichtig
- Gutes Design bleibt beim Entwickler



Pattern

- Verwendung der richtigen Pattern und Namen
- Erstellung von testbarem Code



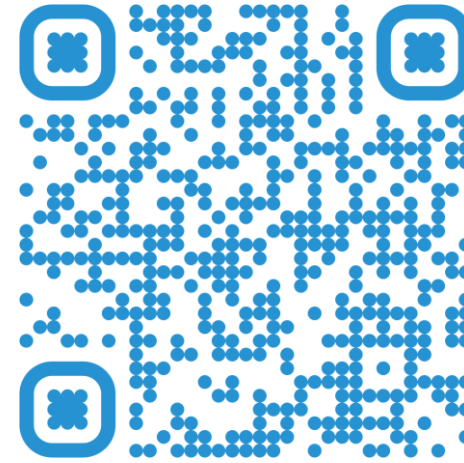
Testen

- Unit Test als Mindeststandard
- Regelmäßige Testläufe zur Prüfung
- Schnelle Weiterentwicklung

Danke



Download



LinkedIn